DBE - plano de integração FNF 0.8.7

Nome: DBE (Debug Engine)

Base de engenharia: Codename Engine 1.1.0, Haxe 4.3.7

Base de jogo pretendida: Friday Night Funkin' 0.8.7

Atualizado: 6 de outubro de 2026

Objetivo e estado atual

A DBE será a engine anfitriã. A arquitetura de build, mods, scripts, editores e depuração da CNE continua sendo a fundação; sistemas de jogo e comportamento do FNF 0.8.7 serão portados por subsistemas e verificados contra fixtures. O resultado deve priorizar inspeção e modificação durante o desenvolvimento, preservando compatibilidade quando um contrato legado puder ser mantido.

Esta implementação é uma primeira entrega incremental. A integração do jogo FNF 0.8.7, a paridade integral das APIs Psych, o UI Editor completo e os adaptadores completos de UI ainda não foram concluídos. O checkout DBE não tem diretório Git, então ainda não existe uma base versionada nem um histórico de alterações para revisar. O diretório `Funkin-main/assets` está vazio porque os assets são submódulos separados. Nenhum asset FNF foi copiado para a DBE.

Mudanças concretas entregues nesta etapa:

- Um conversor de mod em pasta/ZIP, com normalização de wrappers e caminhos comuns, preservação da origem e manifesto por arquivo: `tools/dbe_mod_converter.py`.
- Extensão `.hxc` encaminhada ao HScript Improved e anunciada pela consulta de capacidades. Isso permite que a engine tente carregar a extensão; não implementa a classe-base, os imports ou a API Polymod do V-Slice.
- Exportadores do UI Builder com escape específico para HScript/Lua/XML e rejeição de identificadores/câmeras HScript inválidos.
- Schema 2 do documento UI com migração de documentos planos sem versão/schema 1, preservação de UIDs e validação de referências pai, órfãos e ciclos; a árvore visual depth-first e a escolha do pai no inspetor agora usam esse modelo. Posições continuam absolutas no canvas, sem transformações ou layout herdado.
- Título do aplicativo com DBE e `engine.displayName` para scripts. `engine.name`, namespaces, caminhos de save e APIs CNE são mantidos por compatibilidade.

Arquitetura e restrições

- Manter `project.xml`, o runtime de jogo e o fluxo de build da DBE. O FNF usa um build HXP diferente; não substituir a configuração da DBE por cópia integral do build do FNF.
- Portar gameplay, menus, charts, notas, personagens, stages, eventos, áudio, input e dados em mudanças separadas. Cada subsistema precisa de fixture, teste de regressão e compatibilidade com os hooks existentes antes de avançar.
- Não compilar ou distribuir assets oficiais do FNF no repositório público. O checkout de assets deve seguir o fluxo e a licença publicados pelo projeto oficial e ser instalado separadamente pelo usuário.
- A README da CNE nesta árvore pede que autores consultem a equipe antes de publicar seu código como uma engine derivada separada. Resolver essa condição, atribuições e avisos de cada dependência antes de qualquer distribuição pública da DBE.
- Construções Lua dependem do LuaJIT nativo e do define `LUA_ALLOWED`; a configuração validada nesta etapa é Windows. Linux/macOS ainda precisam de build e smoke test.
- Scripts executam no processo do jogo. O plano não trata Lua/HScript como sandbox e não promete limites de CPU ou memória.

Etapas de implementação

0. Proveniência e linha de base

- Criar checkout versionado para o código da DBE e registrar o estado atual como base; confirmar antes que assets, saídas locais e dependências geradas não sejam adicionados acidentalmente.
- Registrar versão, fonte, licença, dependências, defines e alvos dos checkouts DBE, FNF e Psych usados como referência.
- Obter assets FNF por uma instalação separada, nunca copiando conteúdo protegido para o código público.
- Critério de saída: build inicial registrado, diferenças futuras revisáveis e procedimento reproduzível de recuperação.

1. Portabilidade do jogo base

- Mapear classes, dados e rotas de estado do FNF 0.8.7 para o runtime DBE antes de portar cada subsistema.
- Integrar primeiro carregamento de dados, músicas, charts e gameplay; depois menus, personagens/stages, eventos, diálogo, input e telas auxiliares.
- Preservar APIs e caminhos de mods existentes. Quando a mudança for incompatível, fornecer alias/migração e documentar a versão de origem.
- Critério de saída: regressão funcional das músicas, charts, personagens, stages e fluxos equivalentes ao FNF 0.8.7 em Windows, Linux e macOS, com assets instalados localmente.

2. Contratos de scripting e Lua

- Definir perfis de origem explícitos para Psych 0.6.3 e Psych 1.0.4, e catalogar por versão nome, assinatura, padrões, retorno, efeitos colaterais, callback, ordem e regra de cancelamento.
- Transformar os 246 binds Psych já encontrados em testes de contrato; contagem de nomes não é evidência de equivalência funcional.
- Fechar as divergências registradas: scheduler/índices de notas multi-strumline, `onSpawnNote`, diálogos e retratos, configurações persistentes, efeitos de conquistas e `runHaxeCode`.
- Manter HScript CNE. `.hxc` será tratado como extensão de fonte HScript até que fixtures demonstrem as classes e APIs do V-Slice que também precisam ser portadas.
- Critério de saída: fixtures versionadas dos dois perfis cobrem assinaturas e comportamentos; toda exceção conhecida aparece na matriz sem alegação universal de compatibilidade.

3. Conversor de hierarquia de mods

- Aceitar pastas e ZIPs; detectar Psych, CNE ou V-Slice com base em metadados conhecidos ou exigir `--source-format` quando ambíguo.
- Criar diretório DBE novo e copiar o conteúdo; preservar a origem e recusar sobrescrever destinos existentes.
- Normalizar apenas wrappers e caminhos conhecidos. Não converter scripts arbitrários nem prometer equivalência de gameplay.
- Escrever `_dbe_conversion_report.json` com arquivos convertidos, preservados, ignorados e incompatíveis, dependências, avisos e possíveis perdas.
- Rejeitar traversal, caminhos absolutos, links simbólicos, corrupção, nomes que colidem sem distinção de caixa e pacotes excessivos antes de confirmar a saída.
- Critério de saída: fixtures reais por origem carregam na DBE ou têm diferenças explicadas no manifesto.

4. UI Editor e adaptadores de UI

- Evoluir `UIBuilderData` e `UIBuilderEditor`, usando os assets e controles da CNE. A especificação arquitetural de referência é `docs/ENGINE-INITIAL-AUDIT.md`.
- Migrar o schema plano atual para documento versionado com hierarquia, componentes, layout responsivo, estados, eventos e assets; preservar UIDs e campos legados.
- Ligar viewport, hierarchy e inspector à mesma seleção/modelo. Adicionar transações de undo/redo, dirty state, salvar, autosave, recuperação e preview usando a semântica de runtime do documento.

- Criar importadores e exportadores por adaptador para Psych Lua, CNE HScript e V-Slice. Importar somente padrões reconhecidos; registrar campos que o destino não representa e manter fonte não convertida.
- Depois de estabilizar o modelo, concluir workspaces/docking, temas, atalhos, busca, asset browser e timeline/eventos.
- Critério de saída: round-trip por fixture, preview carregável e compilação/execução no engine-alvo sem perda silenciosa.

5. Depuração e controle de runtime

- Estender console/inspector existentes com registro comum de propriedades, eventos e hooks para estados, menus, gameplay, HUD, input, áudio, câmera, assets, saves, mods e scripts.
- Permitir inspeção e edição em runtime; registrar origem, arquivo, perfil, callback e stack/linha quando disponíveis.
- Expor extensões por APIs/hooks. Alterar código central compilado continua exigindo recompilação.
- Critério de saída: cada superfície listada tem um caminho documentado de inspeção/edição e smoke test de runtime. Medir responsividade e investigar gargalos observados, sem exigir micro-otimização como meta.

6. Licença, distribuição e suporte

- Publicar matriz de compatibilidade e limitações, instruções de instalação externa de assets e fluxo do conversor.
- Revisar licença/atribuição de CNE, FNF, Psych, dependências nativas, fonts, áudio e imagens. Resolver a orientação de fork da CNE antes de publicar como engine independente.
- Fazer build/teste por alvo desktop e documentar dependências e diferenças de plataforma. Não declarar Linux/macOS prontos antes de executá-los.

Validação

- **Build e runtime:** compile DBE em Windows, Linux e macOS; teste inicial sem conteúdo FNF externo e jogo completo com os assets locais instalados pelo usuário.
- **Gameplay:** cobrir menus, seleção, chart, notas, saúde/score, personagens, stages, eventos, áudio, input, pausa, game over e save.
- **Psych Lua:** executar uma suíte por versão que valida nomes, parâmetros, retornos, efeitos, ordem, cancelamento e erros para callbacks e bindings; rodar mods de fixture no gameplay.
- **UI Editor:** cobrir migração e round-trip, hierarquia, seleção, undo/redo, save/reload, autosave/recovery, canvas portrait/landscape, preview e adaptadores de destino.
- **Conversor:** testar pasta/ZIP, wrapper único, metadados ausentes/inválidos, arquivos desconhecidos, colisões de caixa, traversal, ZIP corrompido, paths reservados de Windows, relatório preexistente e imutabilidade da origem.
- **Aceite global:** nenhum arquivo ou campo incompatível é descartado sem registro; a saída do conversor não sobrescreve origem/destino; todas as declarações de suporte apontam para teste reproduzível.

Evidência desta etapa

- `pwsh -File building/compat-test.ps1 -BuildEngine` passou: testes Haxe de hooks, dispatch, diálogos, property paths, capacidades e UI Builder (migração schema 0/1 para 2, validação e ordem da árvore), runtime LuaJIT nativo e build release Windows.
- `python -m unittest discover -s tests -p 'test_mod_converter.py' -v` passou em 10 casos do conversor.
- `python -m compileall -q tools/dbe_mod_converter.py tests/test_mod_converter.py` passou.
- Nenhum teste de jogo com assets FNF foi possível neste checkout vazio de submódulo. Linux/macOS continuam sem validação.

Referências no checkout

- `docs/ENGINE-INITIAL-AUDIT.md` e `docs/compatibility/` - arquitetura do UI Editor, contratos Lua e lacunas atuais.
- `docs/compatibility/MOD-CONVERTER.md` - uso e limites do conversor; `Funkin-main/project.hxp` - versão 0.8.7.
- `Funkin-main/.gitmodules` e `Funkin-main/docs/COMPILING.md` - submódulos externos e instruções de build/assets.